IEFP – Instituto de Emprego e Formação profissional Viseu

Ação: **Aprend+ 02/2025**

UFCD**: 10797**

Gestão e Armazenamento de Dados

Tarefa

BD - Caminhadas Pedestres

Formador: Rui Pereira

Viseu, 18/11/2025

Índice

1. INTRODUÇÃO 4

1.1. MODELO ER (ENTIDADE RELACIONAR) 4

1.2. DATABASE DIAGRAM 5

1.3. CRIAÇÃO DE TABELAS 6

1.3.1. Tabela Estados Conservação 7

1.3.2. Tabela Tipos de Objetos 7

1.3.3. Tabela Nível Dificuldade 8

1.3.4. Tabela Objetos 8

1.3.5. Tabela Atividades 9

1.3.6. Tabela Percursos 9

1.3.7. Tabela Guias Pedestres 10

1.3.8. Tabela Caminhadas 11

1.3.9. Tabela Atividade Percurso 11

1.3.10. Tabela Pessoas Inscritas 12

1.3.11. Tabela Inscritas 12

1.3.12. Tabela Percursos Objetos 13

1.3.13. Tabela Utilizadores 13

1.4. INSERTS 14

1.4.1. Inserts Estados Conservação 14

1.4.2. Inserts Tipos de Objetos 14

1.4.3. Inserts Nível Dificuldade 14

1.4.4. Inserts Atividades 14

1.4.5. Inserts Objetos 15

1.4.6. Inserts Percursos 15

1.4.7. Inserts Inscritas 16

1.4.8. Inserts Pessoas Inscritas 17

1.4.9. Inserts Percursos Objetos 18

1.4.10. Inserts Guias Pedestres 18

1.4.11. Caminhadas 19

1.4.12. Atividade Percurso 19

1.5. PESQUISAS 20
1.5.1. Pesquisa 1 ..... *20*
1.5.2. Pesquisa 2 ..... *21*
1.5.3. Pesquisa 3 ..... *21*
1.5.4. Pesquisa 4 ..... *22*
1.6. VISTAS 23
1.6.1. Vista 1 ..... *23*
1.6.2. Vista 2 ..... *23*

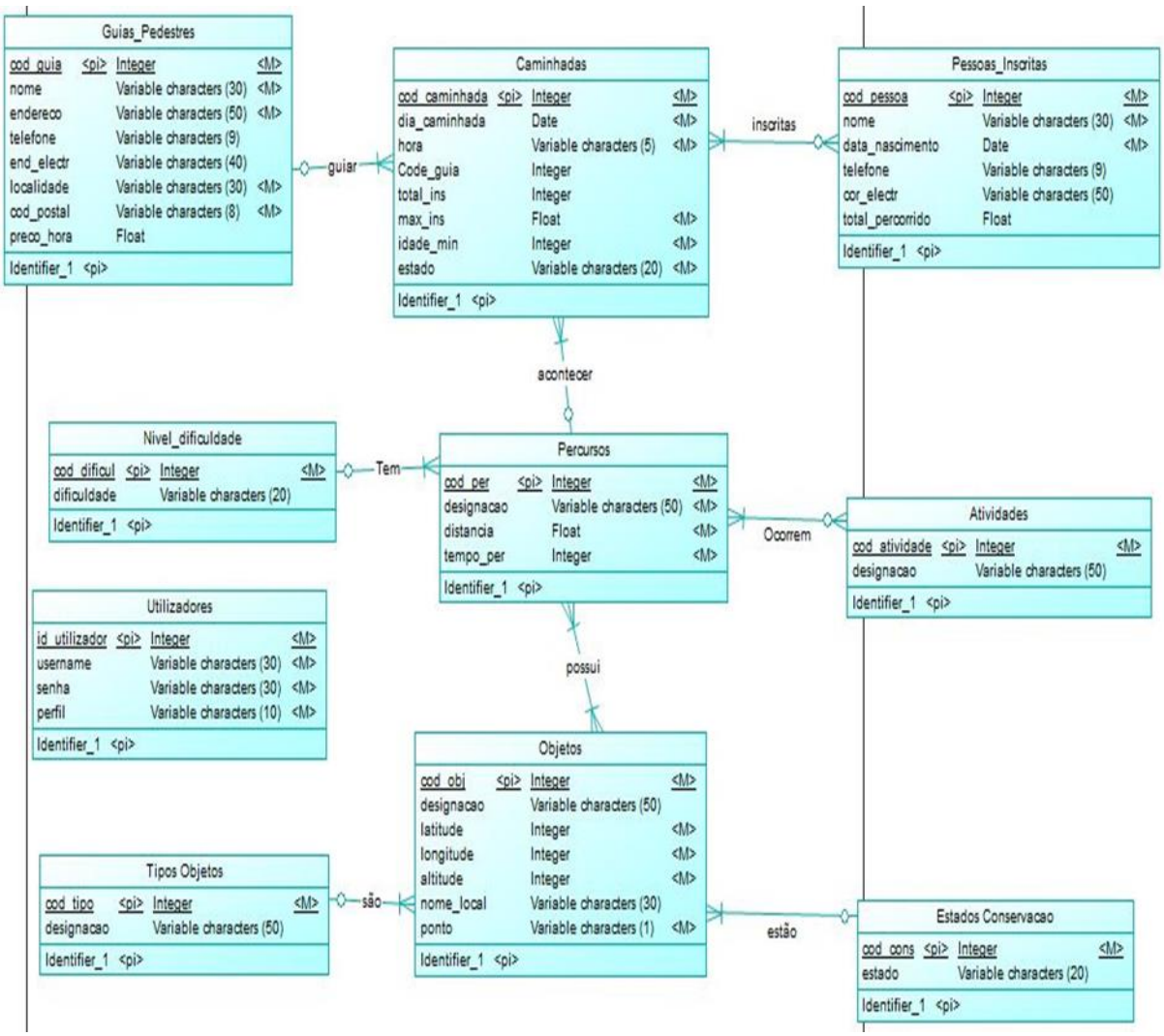
1. Introdução

O objetivo deste trabalho tem como base o desenvolvimento de uma **base de dados** para a Camara Municipal de Viseu, com o intuito de gerir percursos e organizar caminhadas.

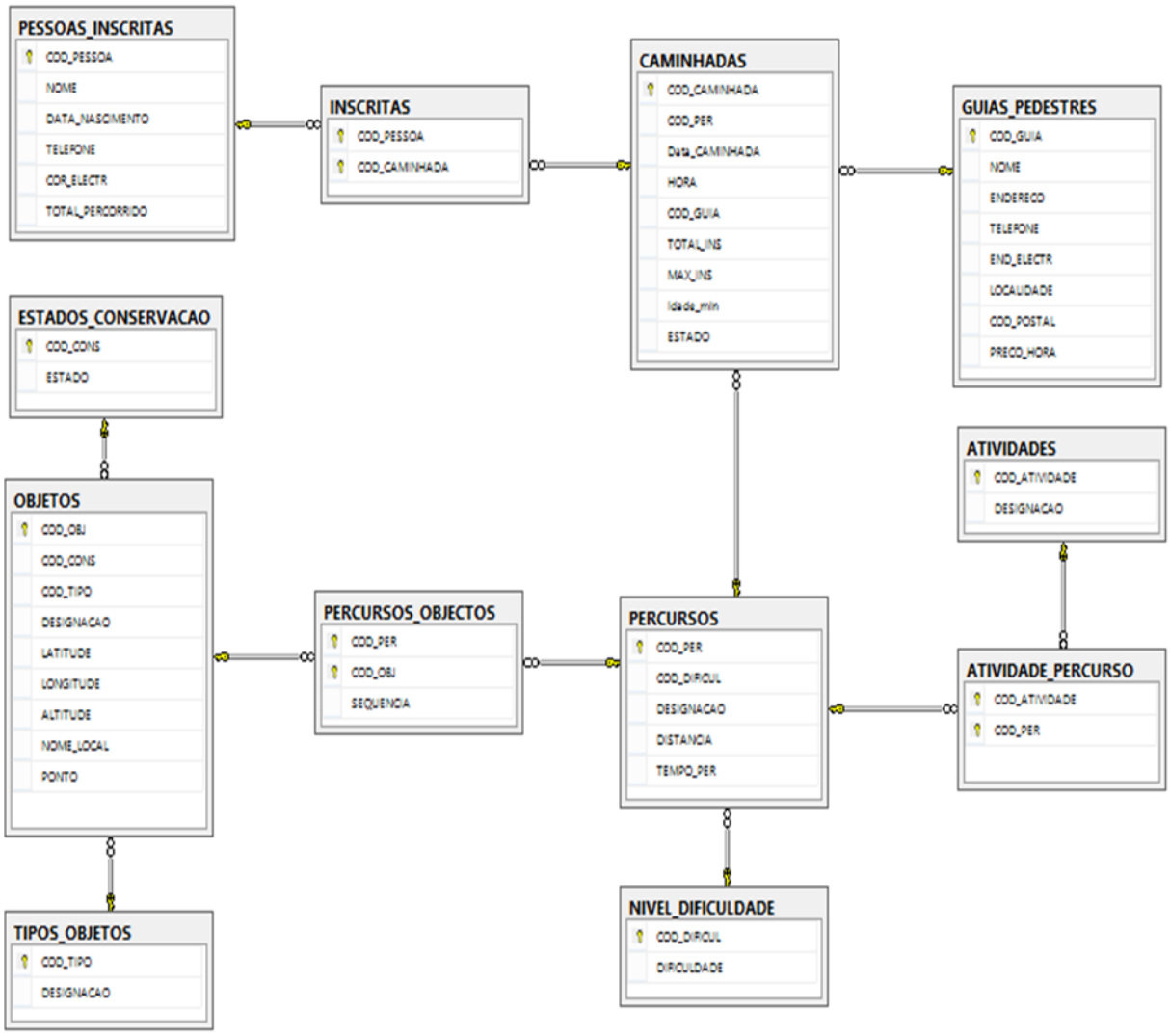
Os formandos a seguir deverão produzir as tabelas que fazem parte da base de dados.

Nota: De salientar que a base de dados proposta pelo formador, pode ser objeto de modificação, isto é, se achar pertinente pode acrescentar ou modificar o modelo apresentado.

1.1. Modelo ER (Entidade Relacionar)



1.2. Database Diagram



1.3. Criação de Tabelas

Elabore um Script para criar a base dado **CAMINHADAS_PEDRESTRES** de acordo com o diagrama acima apresentado.

Nota: o script deve criar todas as tabelas de uma só vez!

```
/*****************************************************************/
/*                        CREATE DATABASE                        */
/*****************************************************************/
GO

USE master

IF DB_ID('CAMINHADAS_PEDESTRES') IS NOT NULL
DROP DATABASE CAMINHADAS_PEDESTRES
CREATE DATABASE CAMINHADAS_PEDESTRES

GO

USE CAMINHADAS_PEDESTRES

/*===============================================================*/
/* DROP TABLE SE EXISTIREM (IF OBJECT_ID)                        */
/*===============================================================*/

IF OBJECT_ID('PERCURSOS_OBJECTOS','U') IS NOT NULL
   DROP TABLE PERCURSOS_OBJECTOS
```

1.3.1. Tabela Estados Conservação

Esta tabela guarda os diferentes estados de conservação dos objetos (fonte, sinalização, banco jardim, etc.), esta tabela apenas foi criada para evitar a redundância de informação na tabela objetos, poupando assim espaço em memória.

```
CREATE TABLE ESTADOS_CONSERVACAO
(
   COD_CONS          INT           NOT NULL,
   ESTADO            VARCHAR(20) DEFAULT NULL,

   CONSTRAINT PK_ESTADOS_CONSERVACAO PRIMARY KEY (COD_CONS)
);
```

1.3.2. Tabela Tipos de Objetos

Tabela que armazena os diferentes tipos de objetos como por exemplo, banco, fonte, setas direcionais, etc.

Esta tabela foi criada com o mesmo prepósito que a anterior.

```
CREATE TABLE TIPOS_OBJETOS
(
   COD_TIPO          INT           NOT NULL,
   DESIGNACAO VARCHAR(50)          NOT NULL,

   CONSTRAINT PK_TIPOS_OBJETOS PRIMARY KEY (COD_TIPO)
);
```

1.3.3. Tabela Nível Dificuldade

Tal como as anteriores esta tabela também é uma tabela auxiliar, apenas armazena o código da dificuldade e a sua designação, por exemplo difícil, fácil, etc. Esta tabela será posteriormente usada na tabela percursos como veremos mais tarde.

```
CREATE TABLE NIVEL_DIFICULDADE
(
   COD_DIFICUL       INT           NOT NULL,
   DIFICULDADE       VARCHAR(20)   NOT NULL,

   CONSTRAINT PK_NIVEL_DIFICULDADE PRIMARY KEY (COD_DIFICUL)
);
```

1.3.4. Tabela Objetos

Os objetos são componentes dos nossos percursos, podendo estes estar em diferentes estados de conservação (ver Tabela Estados de Conservação) bem como de diferentes tipos, como foi falado anteriormente (Tabela Tipos de Objetos), nesta tabela também é armazenado as coordenadas onde se situa cada objeto, bem como o local. Um **ponto** no percurso indica que existe uma mudança de direção, e sempre que ocorre uma mudança terá de existir um objeto, por exemplo uma indicação, sempre que isso acontece atribui-se no atributo ponto um 's' se este for ponto e um 'n' se este não for ponto.

```
CREATE TABLE OBJETOS
(
   COD_OBJ              INT                          NOT NULL,
   COD_CONS             INT                          NOT NULL  ,
   COD_TIPO             INT                          NOT NULL        ,
   DESIGNACAO           VARCHAR(50) DEFAULT NULL,
   LATITUDE             FLOAT                        NOT NULL,
   LONGITUDE            FLOAT                        NOT NULL,
   ALTITUDE             INT                          NOT NULL,
   NOME_LOCAL           VARCHAR(30),
   PONTO                VARCHAR(1)                   NOT NULL,

   CONSTRAINT PK_OBJETOS_COD_OBJ  PRIMARY KEY (COD_OBJ),
   CONSTRAINT FK_OBJETOS_COD_CONS FOREIGN KEY (COD_CONS) REFERENCES
ESTADOS_CONSERVACAO(COD_CONS),
   CONSTRAINT FK_OBJETOS_COD_TIPO FOREIGN KEY (COD_TIPO) REFERENCES
TIPOS_OBJETOS(COD_TIPO)
);
```

1.3.5. Tabela Atividades

Existem diversos tipos de atividades, por exemplo uma caminhada, uma visita, etc. Esta tabela é utilizada essencialmente para evitar a repetição de dados.

```
CREATE TABLE ATIVIDADES
(
   COD_ATIVIDADE     INT                 NOT NULL,
   DESIGNACAO        VARCHAR(50),

   CONSTRAINT PK_ATIVIDADES PRIMARY KEY (COD_ATIVIDADE)
);
```

1.3.6. Tabela Percursos

Esta tabela armazena os dados dos percursos pedestres, cada percurso é avaliado com um nível de dificuldade, tem uma distância e também é-lhe atribuído uma designação (Nome do percurso).

```
CREATE TABLE PERCURSOS
(
   COD_PER       INT         NOT NULL,
   COD_DIFICUL   INT         NOT NULL,
   DESIGNACAO    VARCHAR(50) NOT NULL,
   DISTANCIA     FLOAT       NOT NULL,
   TEMPO_PER     INT         NOT NULL,

   CONSTRAINT PK_PERCURSOS                    PRIMARY KEY (COD_PER),
   CONSTRAINT FK_PERCURSOS_COD_DIF            FOREIGN KEY (COD_DIFICUL) REFERENCES
NIVEL_DIFICULDADE (COD_DIFICUL),
   CONSTRAINT CHK_PERCURSOS_DISTANCIA    CHECK (DISTANCIA>0)

);
```

1.3.7. Tabela Guias Pedestres

A Camara Municipal de Viseu organiza atividades nas quais é necessário contratar um guia pedestres, os dados pessoais desses profissionais são armazenados nesta tabela, bem como o custo de uma hora de trabalho (PRECO_HORA).

```
CREATE TABLE GUIAS_PEDESTRES
(
   COD_GUIA          INT          NOT NULL,
   NOME          VARCHAR(30)      NOT NULL,
   ENDERECO      VARCHAR(50)      NOT NULL,
   TELEFONE      VARCHAR(9),
   END_ELECTR    VARCHAR(40),
   LOCALIDADE    VARCHAR(30)      NOT NULL,
   COD_POSTAL    VARCHAR(8)       NOT NULL,
   PRECO_HORA    FLOAT            NOT NULL,

   CONSTRAINT PK_GUIAS_PEDESTRES                    PRIMARY KEY (COD_GUIA),
   CONSTRAINT CHK_GUIAS_PEDESTRES_PRECO_HORA    CHECK (PRECO_HORA>0)
);
```

1.3.8. Tabela Caminhadas

Como já foi falado no tópico anterior a Camara Municipal frequentemente organiza caminhadas, estas são realizadas num percurso predefinido no sistema (Tabela Percursos), e é definida uma data e hora, a duração prevista para a caminhada, o total de pessoas já inscritas e o máximo admissível de inscrições. Existe também uma idade mínima para uma dada pessoa se poder inscrever ou não em cada caminhada.

```
CREATE TABLE CAMINHADAS
(
   COD_CAMINHADA        INT IDENTITY(1,1)        NOT NULL,
   COD_PER              INT                      NOT NULL,
   Data_CAMINHADA       DATE DEFAULT GETDATE()   NOT NULL,
   HORA                 VARCHAR(5)               NOT NULL,
   COD_GUIA             INT                      NOT NULL,
   TOTAL_INS            INT DEFAULT 0            NOT NULL,
   MAX_INS              INT                      NOT NULL,
   Idade_min            INT                      NOT NULL,
   ESTADO               BIT DEFAULT 0            NOT NULL,

   CONSTRAINT PK_CAMINHADAS                      PRIMARY KEY (COD_CAMINHADA),
   CONSTRAINT FK_CAMINHADAS_PERCURSO     FOREIGN KEY (COD_PER) REFERENCES
PERCURSOS(COD_PER),
   CONSTRAINT FK_CAMINHADAS_GUIA         FOREIGN KEY (COD_GUIA) REFERENCES
GUIAS_PEDESTRES(COD_GUIA),

   CONSTRAINT CHK_CAMINHADAS_MAX_INS CHECK (MAX_INS>0),
   CONSTRAINT CHK_CAMINHADAS_TOTAL_MAX CHECK (MAX_INS>=TOTAL_INS)
);
```

1.3.9. Tabela Atividade Percurso

Esta tabela é criada com o prepósito de "ligar" as tabelas atividade e percursos, pois estas têm uma ligação de N para M, ambos os campos desta tabela são chaves estrangeiras, das tabelas referidas.

```
CREATE TABLE ATIVIDADE_PERCURSO
(
   COD_ATIVIDADE INT       NOT NULL,
   COD_PER       INT       NOT NULL,

   CONSTRAINT PK_ATIVIDADE_PERCURSO      PRIMARY KEY (COD_ATIVIDADE, COD_PER),
   CONSTRAINT FK_A_PERCURSO_COD_ACT      FOREIGN KEY (COD_ATIVIDADE) REFERENCES
ATIVIDADES(COD_ATIVIDADE),
   CONSTRAINT FK_A_PERCURSO_COD_PER      FOREIGN KEY (COD_PER) REFERENCES
PERCURSOS(COD_PER)
);
```

1.3.10. Tabela Pessoas Inscritas

Esta tabela armazena os dados pessoais das pessoas inscritas no sistema, as pessoas que queiram participar numa caminhada têm que ser adicionadas anteriormente a esta tabela.

```
CREATE TABLE PESSOAS_INSCRITAS
(
   COD_PESSOA              INT    IDENTITY (1,1)      NOT NULL,
   NOME                    VARCHAR(30)                NOT NULL,
   DATA_NASCIMENTO         DATE                       NOT NULL,
   TELEFONE                VARCHAR(9)    DEFAULT 'Não tem',
   COR_ELECTR              VARCHAR(50)   DEFAULT 'Não tem',
   TOTAL_PERCORRIDO        FLOAT DEFAULT 0,

   CONSTRAINT PK_PESSOAS_INSCRITAS PRIMARY KEY (COD_PESSOA)
);
```

1.3.11. Tabela Inscritas

Nesta tabela são armazenadas as inscrições ou seja as pessoas inscritas nas diferentes caminhadas, esta tabela complementa uma ligação M para N entre as pessoas inscritas e a tabela caminhada.

```
CREATE TABLE INSCRITAS
(
   COD_PESSOA         INT         NOT NULL,
   COD_CAMINHADA      INT         NOT NULL,

   CONSTRAINT PK_CAMINHADAS_PESSOAS            PRIMARY KEY (COD_PESSOA,COD_CAMINHADA),
   CONSTRAINT FK_INSCRITAS_COD_PESSOA    FOREIGN KEY (COD_PESSOA)    REFERENCES
PESSOAS_INSCRITAS(COD_PESSOA),
   CONSTRAINT FK_INSCRITAS_COD_CAM             FOREIGN KEY (COD_CAMINHADA)REFERENCES
CAMINHADAS(COD_CAMINHADA)
);
```

1.3.12. Tabela Percursos Objetos

Esta tabela "junta" a tabela Percursos e a Tabela Objetos, já que um percursos tem vários objetos e um objeto poderá pertencer a vários percursos, isso acontece quando os percursos de sobrepõem, ou cruzam.

```
CREATE TABLE PERCURSOS_OBJECTOS
(
   COD_PER          INT          NOT NULL,
   COD_OBJ          INT          NOT NULL,
   SEQUENCIA        INT          NOT NULL,

   CONSTRAINT PK_PERCURSOS_OBJECTOS PRIMARY KEY (COD_PER,COD_OBJ),
   CONSTRAINT FK_PER_OBJ_COD_PER FOREIGN KEY (COD_PER) REFERENCES PERCURSOS(COD_PER),
   CONSTRAINT FK_PER_OBJ_COD_OBJ FOREIGN KEY (COD_OBJ) REFERENCES OBJETOS(COD_OBJ)
);
```

1.3.13. Tabela Utilizadores

Esta tabela apenas é criada para fazer o login na aplicação e para distinguir os administradores dos outros utilizadores.

```
CREATE TABLE UTILIZADORES
(
       ID_UTILIZADOR INT                NOT NULL,
       USERNAME            VARCHAR(30) NOT NULL,
       SENHA               VARCHAR(30) NOT NULL,
       PERFIL              VARCHAR(10) NOT NULL,

       CONSTRAINT PK_Utilizadores_ID_UTILIZADOR PRIMARY KEY (ID_UTILIZADOR)
);
```

1.4. Inserts

Para inserir registos na base de dados, deverá criar o Script de acordo com os dados apresentados abaixo.

Nota: Atenção à ordem de inserção dos dados para não haver conflitos de integridade referencial!

1.4.1. Inserts Estados Conservação

	COD_CONS	ESTADO
1	1	Bom Estado
2	2	Mau Estado
3	3	Inexistente

1.4.2. Inserts Tipos de Objetos

	COD_TIPO	DESIGNACAO
1	1	Virar à direita
2	2	Virar à esquerda
3	3	Sentido errado
4	4	Caixote do lixo
5	5	Banco
6	6	Fonte
7	7	Seta em frente

1.4.3. Inserts Nível Dificuldade

	COD_DIFICUL	DIFICULDADE
1	1	Facil
2	2	Medio
3	3	Dificil

1.4.4. Inserts Atividades

	COD_ATIVIDADE	DESIGNACAO
1	1	Caminhada
2	2	Corrida
3	3	Visita
4	4	Lazer
5	5	Convivio

1.4.5. Inserts Objetos

	COD_OBJ	COD_CONS	COD_TIPO	DESIGNACAO	LATITUDE	LONGITUDE	ALTITUDE	NOME_LOCAL	PONTO
1	1	1	1		40,6	8	200	Repeses	s
2	2	1	1		40,6	7,9	170	Jugueiros	s
3	3	2	2		42,2	6,9	170	Quinta do Galo	s
4	4	1	2		39,1	7,2	200	Boquinhas	s
5	5	3	3		41,2	6,4	190	Rossio	n
6	6	2	7		41	8,1	198	Centro	s
7	7	2	2		41	8,3	174	Abravesses	n
8	8	2	6	Água amarela	40,4	8	190	Solar dos Galos	n
9	9	1	2		41,3	7,1	188	Quinta do galo	s
10	10	1	5		43,6	8,1	178	IPVcity	s
11	11	2	5	Banco danificado	39	7,6	198	Quinta do Bispo	s
12	12	1	1		40,9	8,1	200	rua tirilene	s
13	13	1	1		39,2	7,5	201	Adro da Sé	s
14	14	1	5		37,5	8,4	199	Cava	n
15	15	3	4	Ecoponto	40,6	7,5	183	Marzovelos	n
16	16	1	4	Ecoponto	41,3	7,8	170	Largo do pastor	n
17	17	1	5		39,1	6,9	184	Forum	n
18	18	2	6	Água nao potavel	37,5	8,1	170	Lagar das Sopas	n
19	19	2	6	Sem água	39	8	182	Quinta Solar	s
20	20	1	5		32	7,4	177	Travessa Fechada	n

1.4.6. Inserts Percursos

	COD_PER	COD_DIFICUL	DESIGNACAO	DISTANCIA	TEMPO_PER
1	1	1	Familiar	2,2	1
2	2	3	Cultural	8,5	4
3	3	2	Adegas	4,3	3
4	4	3	Montanha	10,5	7
5	5	2	Parque	2,9	2

1.4.7. Inserts Inscritas

	COD_PESSOA	COD_CAMINHADA
1	1	1
2	1	3
3	1	11
4	1	17
5	2	1
6	2	2
7	2	3
8	2	7
9	2	19
10	3	1
11	3	5
12	3	8
13	3	10
14	3	13
15	3	16
16	4	1
17	4	5
18	4	8
19	4	11
20	4	14
21	4	16
22	4	20
23	5	2
24	5	6
25	5	7
26	5	9
27	5	14
28	5	18
29	6	2
30	6	6
31	6	12
32	6	15
33	6	18
34	7	2
35	7	6
36	7	9
37	7	12
38	7	15
39	7	18
40	8	3
41	8	9
42	9	3
43	9	10
44	10	3
45	11	4
46	11	7
47	11	11
48	11	13
49	11	15
50	11	19
51	12	4
52	12	8
53	12	10
54	12	13
55	12	15
56	12	19
57	13	5
58	13	14
59	13	17
60	13	20
61	14	5
62	14	12
63	14	14
64	14	20
65	15	6
66	15	8
67	15	14
68	15	17
69	15	20
70	16	6
71	16	9
72	16	12
73	16	15
74	16	18
75	17	6
76	17	11
77	17	13
78	17	19
79	18	7
80	19	7
81	20	7

1.4.8. Inserts Pessoas Inscritas

	COD_PESSOA	NOME	DATA_NASCIMENTO	TELEFONE	COR_ELECTR	TOTAL_PERCORRIDO
1	1	Helena Pais	1990-11-02	741236985	helena@ole.pt	34,3
2	2	Rita Martins	1982-10-10	712356985	rita@ohot.pt	44,5
3	3	Rita Andrade	2005-03-30	111126985	andrade@kj.pt	30
4	4	Francisto Mateus	2001-02-02	945666985	frans@cisco.pt	34,9
5	5	Maria Antoniete	2006-01-27	963258741	antoni@gmail.pt	32,7
6	6	Joaquim Fonseca	1989-11-11	123236985	jsuugf@ole.pt	25,8
7	7	Ana Pilar	1988-10-16	946236985	pilarana@ole.pt	34,7
8	8	Antonio Tome	2007-02-26	364866985	tome@abd.pt	34
9	9	Leonor Braz	1986-04-26	367236985	portugal@ole.pt	36
10	10	Joao Martins	1993-11-08	991236985	hp@ole.pt	15,9
11	11	Joaquim Mendes	1967-12-19	126736985	jafoste@can.pt	24,2
12	12	Filipa Santos	1984-11-05	136926785	agaah@arg.pt	25,5
13	13	João Tomas	1973-01-12	859612458	unskdj@csef.pt	23,5
14	14	Andre Bento	1981-05-15	251485478	sjfn@gdssd.pt	17,9
15	15	Sandra Moitas	1984-08-18	962525002	pjgh@sddfs.pt	10,7
16	16	Goncalo Matias	1984-09-21	857526320	kuwhado@tkjb.pt	17
17	17	Rui Pacheco	1987-04-30	748521547	lnkafje@ikfin.pt	2,9
18	18	Helder Santos	1978-06-25	526352584	alkdh@qkneh.pt	8,5
19	19	Candito Costa	1991-05-07	125212543	njlah@rjtndh.pt	2,9
20	20	Margarida Jesus	1989-03-14	526300521	kjsfh@uuinaj.pt	2,9
21	21	Filipe Almeida	1982-09-23	515424582	jshrb@ksehfh.pt	0
22	22	Joao Braz	1992-11-19	125212543	jbaz@sabdh.pt	0
23	23	Jose Oliveira	1975-05-18	526300521	jopeo@jnp.pt	0

1.4.9. Inserts Percursos Objetos

	COD_PER	COD_OBJ	SEQUENCIA
1	1	1	1
2	1	2	4
3	1	3	3
4	1	4	2
5	1	7	9
6	1	8	5
7	1	17	10
8	1	19	8
9	1	20	11
10	2	3	2
11	2	10	6
12	2	11	5
13	2	12	1
14	2	14	8
15	2	15	7
16	2	17	9
17	2	18	4
18	2	20	3
19	3	1	1
20	3	2	2
21	3	3	3
22	3	5	7
23	3	6	8
24	3	7	4
25	3	9	5
26	3	10	6
27	4	3	2
28	4	8	1
29	4	9	4
30	4	10	4
31	4	12	2
32	4	19	1
33	5	2	5
34	5	5	8
35	5	7	8
36	5	15	7
37	5	20	9

1.4.10. Inserts Guias Pedestres

	COD_GUIA	NOME	ENDERECO	TELEFONE	END_ELECTR	LOCALIDADE	COD_POSTAL	PRECO_HORA
1	1	Raul Xavier	Rua da Foz	978574130	eu@coisas.com	Aveiro	3525-456	4,6
2	2	Eminia	Rua dos Pobres	974562147	erm@inia.com	Viseu	3225-020	5,5
3	3	Joao Paulo	Rua do Coelho	980548961	jptinto@supermax.com	Tondela	3560-096	5,2
4	4	Ema Filipe	Rua do Alveiro	995262260	eflt@vilamoleza.com	Frades	3680-075	4
5	5	Janeca	Av do Brasil	997480208	ja74@dmx.com	Souto	4520-081	4,4

1.4.11. Caminhadas

	COD_CAMINHADA	COD_PER	Data_CAMINHADA	HORA	COD_GUIA	TOTAL_INS	MAX_INS	Idade_min	ESTADO
1	1	3	2013-01-12	14:30	1	4	4	16	1
2	2	4	2013-01-15	14:30	2	4	5	12	1
3	3	2	2013-01-21	8:30	2	5	5	12	1
4	4	1	2013-01-29	8:30	3	2	5	10	1
5	5	2	2013-02-01	8:30	4	4	4	16	1
6	6	1	2013-02-09	14:30	4	6	6	12	1
7	7	5	2013-02-15	14:30	1	6	6	10	1
8	8	3	2013-03-07	8:30	3	5	6	16	1
9	9	2	2013-03-18	14:30	2	4	6	16	1
10	10	1	2013-03-25	8:30	2	3	6	12	1
11	11	2	2013-04-02	14:30	3	4	6	16	1
12	12	4	2013-04-12	14:30	4	4	6	10	1
13	13	3	2013-04-19	14:30	2	4	6	12	1
14	14	3	2013-04-23	8:30	4	5	6	12	1
15	15	2	2013-05-05	14:30	3	5	6	16	1
16	16	1	2013-05-16	8:30	4	2	6	10	1
17	17	5	2013-05-23	8:30	4	3	4	10	1
18	18	3	2013-05-25	14:30	2	4	6	16	1
19	19	2	2013-06-08	14:30	2	4	4	10	0
20	20	1	2013-06-19	8:30	5	4	6	12	0

1.4.12. Atividade Percurso

	COD_ATIVIDADE	COD_PER
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5

1.5. Pesquisas

1.5.1. Pesquisa 1

Devolve o nome do guia, o registo dos ganhos (Preco_Hora * Tempo_percorrido) e total de percursos realizados pelos guias que ganharam abaixo de 80€.

Sugestão (Tabelas: GUIAS_PEDESTRES, CAMINHADAS E PERCURSOS)

Nota: Atenção ao campo **estado** da tabela caminhadas! só devem ser consideradas as caminhadas em que o estado seja igual a 1.

	Nome do guia	Total de ganhos	Total de percursos
1	Ema Filipe	72	6
2	Joao Paulo	62,4	4
3	Raul Xavier	23	2

1.5.2. Pesquisa 2

Devolve o nome dos 3 participantes que mais caminhadas realizaram e o total de km percorridos, tempo global e a velocidade média calculado com base no total de km e tempo global.

Sugestão (Tabelas: PESSOAS_INSCRITAS, INSCRITAS, CAMINHADAS E PERCURSOS e utilize a palavra reservada **TOP** no SELECT EXTERNO!)

-- Syntax for CAST:
CAST (expression AS data_type [(length)])

-- Syntax for CONVERT:
CONVERT (data_type [(length)] , expression [, style])

-- Syntax for ROUND:
ROUND (numeric_expression , length [,function])

	TOP Participantes	Nome Participante	Caminhadas realizadas	Total de km percorridos	Tempo em total	Velocidade média km/h
1	TOP 3	Francisto Mateus	7	34,3	19 h	1,8
2	TOP 3	Ana Pilar	6	44,5	26 h	1,7
3	TOP 3	Filipa Santos	6	30	16 h	1,9

1.5.3. Pesquisa 3

Devolve a localização dos objetos em **mau estado** ou **inexistente**. Mostrar também qual a falha e o tipo de objeto. Ordenar por estado do objeto.

Sugestão (Tabelas: ESTADOS_CONSERVACAO, OBJETOS, TIPOS_OBJETOS e junção **LEFT** OUTER JOIN)

Results | Messages

	Local do objeto	Estado do objeto	Falhas	Tipo de objeto
1	Marzovelos	Inexistente	Ecoponto	Caixote do lixo
2	Solar dos Galos	Mau Estado	Água amarela	Fonte
3	Quinta do Bispo	Mau Estado	Banco danificado	Banco
4	Lagar das Sopas	Mau Estado	Água nao potavel	Fonte
5	Quinta Solar	Mau Estado	Sem água	Fonte

1.5.4. Pesquisa 4

Devolve por grau de dificuldade o nome do guia e o número de caminhadas, o total de participantes bem como a média de participantes. Ordenar por código de dificuldade.

Sugestão (Tabelas: GUIAS_PEDESTRES, CAMINHADAS, PERCURSOS e NIVEL_DIFICULDADE, utilize juncões com **UNION**)

Results | Messages

	Codigo	Grau de dificuldade	Nome do guia	No. Caminhadas	Total participantes	Media participantes
1	3	Dificil	Ema Filipe	2	8	4
2	3	Dificil	Erminia	3	13	4
3	3	Dificil	Joao Paulo	2	9	4
4	2	Medio	Ema Filipe	2	8	4
5	2	Medio	Erminia	2	8	4
6	2	Medio	Joao Paulo	1	5	5
7	2	Medio	Raul Xavier	2	10	5
8	1	Facil	Ema Filipe	2	8	4
9	1	Facil	Erminia	1	3	3
10	1	Facil	Joao Paulo	1	2	2

1.6. Vistas

1.6.1. Vista 1

Devolve os percursos que tiveram (já realizadas) caminhadas lotadas e os percursos que nunca tiveram a lotação esgotada. Mostra também a distância do percurso o tempo e o nível de dificuldade.

Tabelas: NIVEL_DIFICULDADE, PERCURSOS, CAMINHADAS

Results | Messages

	Lotação	Percurso	Distância em km	Tempo em horas	Nível de dificuldade
1	Tiveram lotação max	Adegas	4,3	3	Medio
2	Tiveram lotação max	Cultural	8,5	4	Dificil
3	Tiveram lotação max	Familiar	2,2	1	Facil
4	Tiveram lotação max	Parque	2,9	2	Medio
5	Nunca tiveram lota...	Montanha	10,5	7	Dificil

1.6.2. Vista 2

Devolve por atividade o número de caminhadas realizadas e o número de total de participantes para essa atividade. Por fim mostra o grau de dificuldade associado à atividade. Para o 1º Trimestre de 2013.

Tabelas: ATIVIDADES, ATIVIDADE_PERCURSO, NIVEL_DIFICULDADE, CAMINHADAS, PERCURSOS

Results | Messages

	Atividade	Realizadas	Participantes	Grau dificuldade
1	Comida	3	13	Dificil
2	Lazer	1	4	Dificil
3	Caminhada	3	11	Facil
4	Convivio	1	6	Medio
5	Visita	2	9	Medio

1.6.3. Pesquisa Extra

Listar os objetos que foram utilizados no percurso 1 (COD_PER=1). Listar também os objetos que nunca foram utilizados no percurso 1.

Results | Messages

	DESIGNACAO	Número de Objetos
1	Banco	2
2	Caixote do lixo	0
3	Fonte	2
4	Sentido errado	0
5	Seta em frente	0
6	Virar à direita	2
7	Virar à esquerda	3

1.6.4. Procedimento armazenado

Criar um procedimento armazenado que verifique a inserção de um registo na tabela TIPOS_OBJETOS. Caso o objeto já exista na base de dados, deverá mostrar uma mensagem de erro. ('Já existe um objeto esse código!').

```
/*-----------------------------------------------------------------------------*/
/* PROCEDIMENTO: Verifica a inserção de um registo na tabela TIPOS_OBJETOS.    */
/*-----------------------------------------------------------------------------*/

GO

IF OBJECT_ID('spInserir_TIPOS_OBJETOS') IS NOT NULL
DROP PROC spInserir_TIPOS_OBJETOS

GO

CREATE PROC spInserir_TIPOS_OBJETOS
```

…

```
32 /*-----------------------------------------------------------------------------------*/
33 /*                                    TESTES                                         */
34 /*-----------------------------------------------------------------------------------*/
35
36  GO
37
38 BEGIN TRY
39
40 EXEC spInserir_TIPOS_OBJETOS
41  @COD_TIPO = 8,
42  @DESIGNAÇÃO = 'Banco'
43
44  PRINT 'O registo foi inserido com sucesso.'
45
46  END TRY
47  BEGIN CATCH
48  PRINT 'Ocorreu um erro. O registo não foi inserido.'
49  PRINT 'Número do erro: ' + CONVERT(varchar, ERROR_NUMBER())
50  PRINT 'Mensagem de erro: ' + CONVERT(varchar, ERROR_MESSAGE())
51  END CATCH
52
53  GO
54 BEGIN TRY
55
56 EXEC spInserir_TIPOS_OBJETOS
57  @COD_TIPO = 8,
58  @DESIGNAÇÃO = 'Poste'
59
60  PRINT 'O registo foi inserido com sucesso.'
61  END TRY
62  BEGIN CATCH
63  PRINT 'Ocorreu um erro. O registo não foi inserido.'
64  PRINT 'Número do erro: ' + CONVERT(varchar, ERROR_NUMBER())
65  PRINT 'Mensagem de erro: ' + CONVERT(varchar, ERROR_MESSAGE())
66  END CATCH
67
68  GO
69
70 SELECT * FROM TIPOS_OBJETOS
71
```

1.6.5. Trigger

Opção 1

Criar um trigger ativado para a tabela INSCRITAS quando se tenta inserir uma nova inscrição. Só é inserido um registo se existe a pessoa com o código inserido, a caminhada ainda tenha vagas disponíveis, ainda não tenha sido realizada a caminhada e o registo não seja duplicado. Caso seja possível é inserida a inscrição e depois atualizado o campo TOTAL_INS na tabela CAMINHADAS.

Tabelas: INSCRITAS e UPDATE CAMINHADAS.TOTAL_INS

```
/*******************************************************************************/
/*                                    TRIGGERS                                 */
/*******************************************************************************/

USE CAMINHADAS_PEDESTRES

/*-----------------------------------------------------------------------------*/
/* TRIGGER 1: Activado para a tabela INSCRITAS quando se tenta inserir uma nova      */
/*            inscrição. Só é inserido um registo se existe a pessoa com o código    */
/*            inserido, a caminhada ainda tenha vagas disponíveis, ainda não tenha   */
/*            sido realizada a caminhada e o registo não seja duplicado.             */
/*            Caso seja possível é inserida a inscrição e depois actualizado o campo */
/*            TOTAL_INS na tabela CAMINHADAS.                                        */
/* TABELAS: INSCRITAS e UPDATE CAMINHADAS.TOTAL_INS                                  */
/*-----------------------------------------------------------------------------*/
GO

IF OBJECT_ID('tr_INS_CAMINHADA') IS NOT NULL
DROP TRIGGER tr_INS_CAMINHADA

GO

CREATE TRIGGER tr_INS_CAMINHADA
ON INSCRITAS
```

…

```
68  /*-----------------------------------------------------------------------------*/
69  /*-----------------------------------------------------------------------------*/
70  /*                                    TESTES                                   */
71  /*-----------------------------------------------------------------------------*/
72  /*ANTES TRIGGER*/
73
74  SELECT * FROM CAMINHADAS
75
76  SELECT * FROM INSCRITAS
77
78  /*TESTAR TRIGGER*/
79
80  INSERT INTO INSCRITAS
81  VALUES (5,20)
82
83  INSERT INTO INSCRITAS
84  VALUES (6,19)
85
86  /*DEPOIS TRIGGER*/
87
88  SELECT * FROM CAMINHADAS
89
90  SELECT * FROM INSCRITAS
91  /*-----------------------------------------------------------------------------*/
92  /*-----------------------------------------------------------------------------*/
93
```

Opção 2

Criar um trigger sobre tabela CAMINHADAS quando se faz o UPDATE do ESTADO da caminhada. O trigger atualiza TOTAL_PERCORRIDO na tabela INSCRITAS para as pessoas que realizaram a caminhada. O valor a atualizar está na tabela PERCURSOS no campo DISTANCIA.

Tabelas: UPDATE CAMINHADAS.ESTADO -> UPDATE PESSOAS_INSCRITAS.TOTAL_PERCORRIDO

```
/*-------------------------------------------------------------------------------*/
/* TRIGGER 2: Trigger sobre tabela CAMINHADAS quando se faz o UPDATE do ESTADO da    */
/*            caminhada. O trigger actualiza TOTAL_PERCORRIDO na tabela INSCRITAS    */
/*            para as pessoas que realizaram a caminhada. O valor a actualizar está  */
/*            na tabela PERCURSOS no campo DISTANCIA.                                */
/* TABELAS: UPDATE CAMINHADAS.ESTADO -> UPDATE PESSOAS_INSCRITAS.TOTAL_PERCORRIDO    */
/*-------------------------------------------------------------------------------*/
GO

IF OBJECT_ID('tr_ATC_TOTAL_PER') IS NOT NULL
DROP TRIGGER tr_ATC_TOTAL_PER

GO

CREATE TRIGGER tr_ATC_TOTAL_PER
ON CAMINHADAS
INSTEAD OF UPDATE
```

…..

```
/*-----------------------------------------------------------------------------------*/
/*-----------------------------------------------------------------------------------*/
/*                                        TESTES                                     */
/*-----------------------------------------------------------------------------------*/
/*ANTES TRIGGER*/

SELECT * FROM CAMINHADAS

SELECT * FROM PESSOAS_INSCRITAS

/*TESTAR TRIGGER*/

UPDATE CAMINHADAS
SET ESTADO = 1
WHERE COD_CAMINHADA = 19

UPDATE CAMINHADAS
SET ESTADO = 0
WHERE COD_CAMINHADA = 20

/*DEPOIS TRIGGER*/

SELECT * FROM CAMINHADAS

SELECT * FROM PESSOAS_INSCRITAS

/*-----------------------------------------------------------------------------------*/
/*-----------------------------------------------------------------------------------*/
```

1.6.6. Pergunta desenvolvida pelo formando

Cada formando deverá desenvolver uma consulta, Views, procedimento ou trigger ao seu critério sobre a base de dados Caminhadas pedestres.

Deverá colocar o enunciado e o resultado esperado.